

Laporan Praktikum Kontrol Cerdas

(Praktikum 14 MediaPipe Pose)

Nama : Bujang Setia Aditama
NIM 234308066
Kelas : TKA – 6C
Github : <https://github.com/BujangSA/PROJECT-MEDIAPIPE-HAND>.

Abstrak

Praktikum ini bertujuan untuk mengimplementasikan sistem deteksi dan analisis tangan secara real-time menggunakan library OpenCV dan MediaPipe pada bahasa pemrograman Python. Modul MediaPipe Hands digunakan untuk mendeteksi keberadaan tangan, menampilkan 21 titik landmark beserta garis penghubungnya, serta mengklasifikasikan tangan kanan dan kiri berdasarkan fitur handedness. Sistem bekerja dengan mengambil citra dari webcam, mengonversi format warna dari BGR ke RGB, kemudian memprosesnya untuk menghasilkan data koordinat landmark dalam bentuk nilai ter-normalisasi. Selain itu, dilakukan analisis sederhana untuk membedakan orientasi tangan (telapak atau punggung tangan) dengan membandingkan posisi koordinat ujung jempol dan ujung kelingking. Hasil praktikum menunjukkan bahwa OpenCV dan MediaPipe dapat diimplementasikan secara efektif untuk membangun sistem interaksi berbasis gesture tanpa sensor tambahan. Sistem yang dikembangkan mampu menjadi dasar dalam pengembangan aplikasi Human-Computer Interaction (HCI) berbasis pengenalan gerakan tangan yang lebih kompleks.

Kata kunci: *Computer Vision, MediaPipe Hands, OpenCV, Hand Landmark, Gesture Recognition.*

1. Pendahuluan

Perkembangan teknologi *computer vision* dan *machine learning* memungkinkan sistem komputer untuk memahami gerakan serta struktur tubuh manusia secara real-time. Salah satu solusi yang banyak digunakan untuk analisis gerakan manusia adalah **MediaPipe**, sebuah framework open-source yang dikembangkan oleh Google untuk membangun pipeline pemrosesan data berbasis persepsi (*perception*). Salah satu modul penting di dalamnya adalah **MediaPipe Pose**, yang dirancang untuk mendeteksi dan melacak posisi tubuh manusia secara real-time dari gambar atau video. Bazarevsky, V., et al. (2020).

MediaPipe Pose menggunakan model berbasis *machine learning* yang dikenal sebagai **BlazePose**. Model ini mampu mengidentifikasi hingga 33 titik landmark tubuh manusia, termasuk kepala, bahu, siku, pergelangan tangan, pinggul, lutut, dan pergelangan kaki. Setiap landmark direpresentasikan dalam koordinat tiga dimensi (x, y, z) beserta nilai visibilitas, sehingga memungkinkan analisis postur tubuh yang lebih akurat dibandingkan metode dua dimensi konvensional. Lugaresi, C., et al. (2019).

Keunggulan utama MediaPipe Pose adalah kemampuannya bekerja secara real-time dengan kebutuhan komputasi yang relatif ringan. Framework ini dapat dijalankan pada berbagai platform seperti desktop (Python, C++), Android, iOS, bahkan perangkat berbasis web. Dengan dukungan TensorFlow Lite, proses inferensi model dapat dipercepat menggunakan akselerasi CPU maupun GPU. Hal ini menjadikan MediaPipe Pose banyak dimanfaatkan dalam berbagai bidang, seperti analisis olahraga, rehabilitasi medis, sistem interaksi manusia-komputer (HCI), augmented reality, hingga sistem pemantauan kebugaran. OpenCV Documentation. (2023).

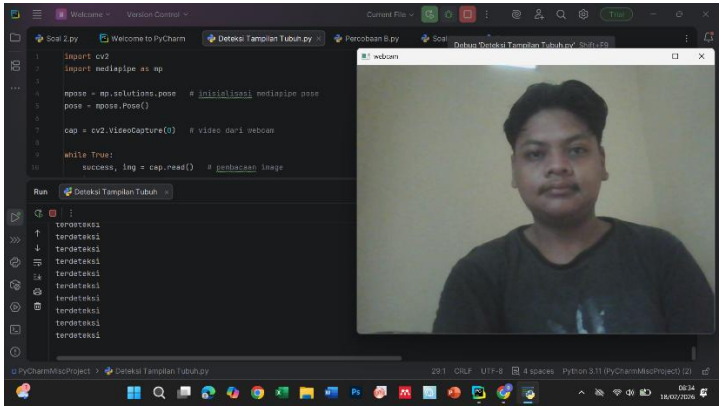
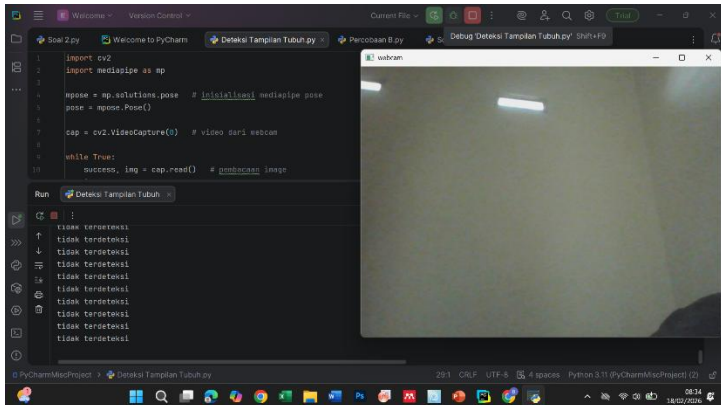
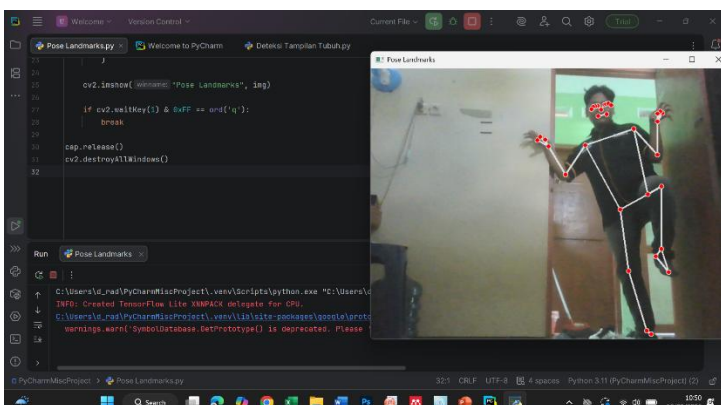
2. Tujuan

- 2.1 Memahami konsep dasar computer vision menggunakan Python.
- 2.2 Memahami konsep mediapipe Pose
- 2.3 Menggunakan MediaPipe Pose untuk mendeteksi pose tubuh manusia secara 2D.
- 2.4 Mengidentifikasi dan mengklasifikasikan yang mencakup bagian mata, hidung, mulut, Pundak, sikut, pergelangan tangan.

3. Manfaat

- 3.1 Meningkatkan pemahaman tentang pemrosesan citra digital dan computer vision.
- 3.2 Meningkatkan pemahaman mediapipe Pose.
- 3.3 Melatih kemampuan pemrograman Python dalam pengolahan data visual.
- 3.4 Memberikan dasar pengembangan sistem berbasis gesture recognition

4. Output hasil running program

No.	Tugas	Hasil dan Dokumentasi
1	Deteksi Tampilan Tubuh	 
2	Pose Landmarks	

5. Analisis program

5.1 Deteksi Tampilan Tubuh

```
Welcome to PyCharm  Pose Landmarks.py  Deteksi Tampilan Tubuh.py x
1  import cv2
2  import mediapipe as mp
3
4  mpose = mp.solutions.pose # inisialisasi mediapipe pose
5  pose = mpose.Pose()
6
7  cap = cv2.VideoCapture(0) # video dari webcam
8
9  while True:
10     success, img = cap.read() # pembacaan image
11     if not success:
12         break
13
14     imgrgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # konversi warna BGR ke RGB
15     hasil = pose.process(imgrgb) # ekstraksi pose
16
17     if hasil.pose_landmarks:
18         print("terdeteksi")
19     else:
20         print("tidak terdeteksi")
21
22     cv2.imshow( winname: "webcam", img) # tampilkan video
23
24     if cv2.waitKey(1) & 0xFF == ord('q'): # tekan q untuk keluar
```

```
24     if cv2.waitKey(1) & 0xFF == ord('q'): # tekan q untuk keluar
25         break
26
27     cap.release()
28     cv2.destroyAllWindows()
29
```

Pada tahap awal dilakukan proses import library OpenCV dan MediaPipe. Library OpenCV digunakan untuk pengolahan citra digital dan pengambilan data dari kamera, sedangkan MediaPipe digunakan sebagai modul pendeteksi tangan. Sumber citra diperoleh dari webcam dengan indeks perangkat 0 melalui perintah `cv2.VideoCapture(0)`, yang kemudian disimpan dalam variabel `cap`. Perintah ini berfungsi untuk mengaktifkan kamera sebagai media input sistem. Selanjutnya dilakukan inisialisasi modul `mp.solutions.pose` yang disimpan dalam variabel `mpPose`. Objek `pose` kemudian dibuat dan disimpan pada variabel `pose`. Objek ini berfungsi sebagai model pendeteksi tangan yang akan memproses citra dan mengidentifikasi keberadaan landmark tangan. Proses utama program berjalan di dalam perulangan `while True`, yang memungkinkan sistem membaca frame video secara terus-menerus (real-time). Setiap iterasi akan mengambil satu frame dari kamera menggunakan `cap.read()`. Jika proses pembacaan frame gagal, maka perulangan akan dihentikan untuk mencegah kesalahan sistem. Frame yang diperoleh dari kamera masih dalam format warna BGR (standar OpenCV). Oleh karena itu,

dilakukan konversi warna ke format RGB menggunakan fungsi `cv2.cvtColor()`, karena MediaPipe memerlukan input dalam format RGB untuk melakukan proses deteksi.

Citra RGB yang telah dikonversi kemudian diproses menggunakan `hands.process(imgRGB)`. Hasil pemrosesan disimpan dalam variabel `results`. Apabila sistem mendeteksi keberadaan tangan, maka atribut `results.multi_hand_landmarks` akan bernilai tidak kosong. Dalam kondisi tersebut, sistem menampilkan teks “Ada Tangan” pada terminal. Sebaliknya, apabila tidak terdapat tangan yang terdeteksi, maka sistem akan menampilkan teks “Tidak Ada Tangan”.

Hasil tampilan kamera ditampilkan secara real-time menggunakan fungsi `cv2.imshow()`. Program akan terus berjalan hingga pengguna menekan tombol ‘q’, yang berfungsi sebagai perintah untuk menghentikan sistem. Setelah proses dihentikan, kamera dilepaskan menggunakan `release()` dan seluruh jendela ditutup menggunakan `destroyAllWindows()` untuk memastikan tidak ada sumber daya yang masih digunakan.

5.2 Pose Landmarks

```
Pose Landmarks.py x Deteksi Tampilan Tubuh.py We
1 import cv2
2 import mediapipe as mp
3
4 mpose = mp.solutions.pose
5 mdraw = mp.solutions.drawing_utils
6
7 pose = mpose.Pose()
8 cap = cv2.VideoCapture(0)
9
10 while True:
11     success, img = cap.read()
12     if not success:
13         break
14
15     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
16     hasil = pose.process(imgRGB)
17
18     if hasil.pose_landmarks:
19         mdraw.draw_landmarks(
20             img,
21             hasil.pose_landmarks,
22             mpose.POSE_CONNECTIONS
23         )
24
25     cv2.imshow(winname: "Pose Landmarks", img)
```

```

25         cv2.imshow( winname: "Pose Landmarks", img)
26
27         if cv2.waitKey(1) & 0xFF == ord('q'):
28             break
29
30     cap.release()
31     cv2.destroyAllWindows()

```

Pada tahap awal dilakukan proses import library OpenCV dan MediaPipe serta inisialisasi kamera menggunakan `cv2.VideoCapture(0)`. Proses ini sama dengan program sebelumnya, yaitu menggunakan webcam sebagai sumber input citra secara real-time.

Selanjutnya dilakukan inisialisasi modul `mediapipe.solutions.hands` yang disimpan dalam variabel `mediapipehand`. Berbeda dengan program sebelumnya, pada program ini ditambahkan parameter `max_num_hands=1` pada saat pembuatan objek `Hands()`. Parameter tersebut berfungsi untuk membatasi jumlah tangan yang dapat dideteksi sistem, yaitu maksimal satu tangan. Pembatasan ini bertujuan untuk meningkatkan fokus deteksi serta mengurangi beban pemrosesan apabila sistem hanya memerlukan satu objek tangan.

Variabel `mpdraw` diinisialisasi dari `mediapipe.solutions.drawing_utils`, yang berfungsi untuk menggambar landmark dan koneksi antar titik tangan pada frame video. Proses pembacaan frame, pengecekan keberhasilan pembacaan, serta konversi warna dari BGR ke RGB sama dengan program sebelumnya. Konversi ini tetap diperlukan karena MediaPipe memproses citra dalam format RGB.

Hasil pemrosesan citra disimpan dalam variabel `results`. Apabila sistem mendeteksi tangan, maka `results.multi_hand_landmarks` akan berisi data landmark tangan. Program kemudian melakukan perulangan untuk setiap tangan yang terdeteksi (meskipun telah dibatasi satu tangan).

Fungsi *mpdraw.draw_landmarks()* digunakan untuk menggambar titik-titik landmark serta garis penghubung antar titik pada tangan. Parameter *mediapipehand.HAND_CONNECTIONS* berfungsi untuk menentukan pola koneksi antar landmark sehingga membentuk struktur tangan yang utuh pada tampilan layar.

Selanjutnya dilakukan perulangan menggunakan `enumerate()` terhadap `titiktangan.landmark`. Setiap landmark memiliki:

`id` → nomor indeks titik (0–20),

`titik.x` → koordinat horizontal (nilai ter-normalisasi 0–1),

`titik.y` → koordinat vertikal (nilai ter-normalisasi 0–1).

Nilai koordinat tersebut dicetak pada terminal. Data ini dapat dimanfaatkan untuk pengembangan lebih lanjut, seperti klasifikasi gesture, perhitungan jarak antar jari, maupun analisis posisi tangan.

6. Kesimpulan

Setelah dilakukan percobaan atau praktikum dapat disimpulkan bahwa :

1. Library OpenCV dan MediaPipe dapat diimplementasikan secara efektif untuk membangun sistem deteksi dan analisis tangan secara real-time menggunakan webcam.
2. Sistem mampu membaca citra dari kamera, mengonversi format warna sesuai kebutuhan model, dan memproses citra untuk mendeteksi anggota tubuh.
3. Program berhasil mendeteksi ada atau tidaknya tangan pada frame video.
4. Data koordinat setiap landmark berhasil diekstraksi dalam bentuk nilai ternormalisasi yang dapat digunakan untuk analisis lanjutan, seperti pengenalan gesture.
5. Sistem dapat mengklasifikasikan tangan sebagai tangan kanan atau tangan kiri berdasarkan fitur handedness dari MediaPipe Pose.
6. Program mampu mengidentifikasi bagian tubuh mencakup mata, hidung, mulut, pundak, sikut, pergelangan tangan dan pergelangan kaki.
7. Secara keseluruhan, sistem yang dikembangkan dapat menjadi dasar pengembangan aplikasi interaksi manusia dan komputer berbasis gesture yang lebih kompleks di masa mendatang.

7. Saran

Berdasarkan hasil praktikum yang telah dilakukan, terdapat beberapa saran untuk pengembangan sistem selanjutnya :

1. sistem dapat ditingkatkan dengan menambahkan metode pengenalan gesture tertentu, seperti gerakan membuka dan menutup tangan atau kombinasi posisi jari, sehingga aplikasi menjadi lebih interaktif.
2. akurasi deteksi dapat ditingkatkan dengan melakukan pengaturan parameter seperti nilai confidence atau dengan menambahkan pencahayaan yang lebih stabil agar hasil deteksi lebih optimal.
3. sistem dapat dikembangkan untuk diintegrasikan dengan perangkat lain, seperti mikrokontroler atau aplikasi berbasis IoT, sehingga gerakan tangan dapat digunakan sebagai media kontrol suatu perangkat.

8. Referensi

- Bazarevsky, V., et al. (2020). *BlazePose: On-device Real-time Body Pose tracking*. Google AI.
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (t.t.). *MediaPipe: A Framework for Building Perception Pipelines*.
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking* (arXiv:2006.10214). arXiv. <https://doi.org/10.48550/arXiv.2006.10214>
- OpenCV Documentation. (2023). *Open Source Computer Vision Library*